Exam Programming for Physics: Physics-Informed Neural Networks to Solve Differential Equations

Vasco Verwimp

28 May 2026

Contents

Chapter 1

Introduction

Traditional approaches to solving differential equations rely on analytical methods or classical numerical techniques such as finite difference methods, finite element methods, and spectral methods. However, these methods often struggle with high-dimensional problems, complex geometries, and inverse problems. In recent years, machine learning has emerged as a powerful alternative for solving differential equations, offering flexibility and scalability that traditional methods cannot provide.

Physics-Informed Neural Networks (PINNs) are a relatively new technique to solve problems described by ordinary or partial differential equations with boundary conditions. The idea gained traction in the late 1990s when several authors discussed solving these equations with a neural network by incorporating the equation directly into the loss function. During this era, the technique remained a niche academic curiosity because computers were not fast enough, and modern libraries (like PyTorch or TensorFlow) did not exist to handle the complex automatic differentiation required.

PINNs are applied to two classical differential equation problems:

1. **The Damped Harmonic Oscillator**: A second-order ordinary differential equation (ODE) representing mechanical vibrations with friction.

2. **Burgers' Equation**: A nonlinear partial differential equation (PDE) that combines advection and diffusion processes.

We investigate how PINNs can solve these equations by training neural networks to simultaneously satisfy the observed data and the underlying physics constraints. This will be used to extrapolate into regions for which no observational data was given. As a benchmark, we will use a standard machine learning model: a simple neural network trained on data loss only. Additionally, we will compare different PINN implementations to assess their predictive capabilities.

1.1 Physics-Informed Neural Networks (PINNs)

A Physics-Informed Neural Network is a neural network that is trained to solve differential equations by incorporating the equation's physics directly into the loss function. The network learns to approximate the solution $u(x, t)$ such that it satisfies both:

1. The observed data (data loss, $\mathcal{L}_{\text{data}}$)

2. The differential equation (physics loss, $\mathcal{L}_{\text{phys}}$)

3. Initial and boundary conditions (constraint loss, $\mathcal{L}_{\text{ic}}$)

The key innovation that has greatly increased the ease of use of PINNs is automatic differentiation to compute derivatives. This allows us to directly enforce the physics constraints encoded in differential equations.

Let $u(x,t)$ denote the solution to a differential equation. A neural network $u_\theta(x,t)$ is trained to approximate this solution, where θ represents the network's parameters. The total loss function is:

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{data}}(\theta) + \lambda_{\text{phys}}\mathcal{L}_{\text{phys}}(\theta) + \lambda_{\text{ic}}\mathcal{L}_{\text{ic}}(\theta) \tag{1.1}$$

where:

- $\mathcal{L}_{\text{data}}$ is the **data loss**: measures the discrepancy between network predictions and observed data

$$\mathcal{L}_{\text{data}} = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \left(u_\theta(x_i,t_i) - u_i^{\text{obs}}\right)^2 \tag{1.2}$$

- $\mathcal{L}_{\text{phys}}$ is the **physics loss**: enforces the differential equation at randomly chosen points, called collocation points, in a certain region.

$$\mathcal{L}_{\text{phys}} = \frac{1}{N_{\text{col}}} \sum_{j=1}^{N_{\text{col}}} \left(\mathcal{F}\left(u_\theta, \frac{\partial u_\theta}{\partial x}, \frac{\partial u_\theta}{\partial t}, \dots\right)\Big|_{x=x_j, t=t_j}\right)^2 \tag{1.3}$$

  where $\mathcal{F}$ represents the differential equation operator: the solution u has $\mathcal{F}(u(x,t))=0$ for $\forall x,t$.

- $\mathcal{L}_{\text{ic}}$ is the **initial/boundary condition loss**: enforces initial and boundary conditions

$$\mathcal{L}_{\text{ic}} = \frac{1}{N_{\text{ic}}} \sum_{k=1}^{N_{\text{ic}}} \left(u_\theta(x_k,t_0) - u_0(x_k)\right)^2 \tag{1.4}$$

- λ_{phys} and λ_{ic} are **loss weights** controlling the trade-off between fitting data and satisfying physics.

1.1.1 Advantages and disadvantages of PINNs

Advantages

+ **No mesh required**: Unlike finite difference or finite element methods, PINNs do not require discretization of the spatial and temporal domains; collocation points can be chosen at random, which limits the discretization effects.

+ **Inverse problems**: Can be used to identify unknown physical parameters from data.

+ **Extendability**: The neural network learns about the workings of the physical system; this allows it to make meaningful predictions in regions that it was not trained for.

Disadvantages

- **Longer compute time**: The added calculation of gradients (via automatic differentiation) significantly increases computational cost compared to standard neural networks.
- **Hyperparameter sensitivity**: Performance is very sensitive to the choice of loss weights (λ_{phys}, λ_{ic}), network architecture, and optimizer settings. Careful tuning is required and can be time-consuming.
- **Discontinuities and shocks**: PINNs struggle with discontinuous solutions and shock fronts because derivatives can become extreme in these cases.

1.1.2 Implementation Details

In practice, a PINN is typically implemented using the following structure:

1. **Network architecture**: A fully-connected feedforward neural network with several hidden layers and activation functions (e.g., tanh, GELU).
2. **Collocation points**: Sample interior collocation points to evaluate the physics residual and boundary/initial condition points separately.
3. **Automatic differentiation**: Using frameworks like PyTorch or TensorFlow to compute any order derivatives.
4. **Optimization**: Train the network using gradient-based optimizers (e.g., Adam, L-BFGS) to minimize the combined loss function.

1.2 The Damped Harmonic Oscillator

1.2.1 Problem Description

The damped harmonic oscillator is one of the most fundamental systems in classical mechanics, describing the motion of a mass attached to a spring with damping. It is governed by the following second-order ordinary differential equation:

$$m\ddot{y}(t) + c\dot{y}(t) + ky(t) = 0 \tag{1.5}$$

where:

- m is the mass $[kg]$
- c is the damping coefficient $[kg/s]$
- k is the spring stiffness $[N/m]$
- $y(t)$ is the displacement $[m]$
- $\dot{y}(t) = \frac{dy}{dt}$ is the velocity $[m/s]$
- $\ddot{y}(t) = \frac{d^2y}{dt^2}$ is the acceleration $[m/s^2]$

The system is typically subject to initial conditions:

$$y(0) = y_0, \quad \dot{y}(0) = v_0 \tag{1.6}$$

1.2.2 Physical Characterization

The behavior of the damped harmonic oscillator can be characterized using two non-dimensional quantities:

The **undamped natural frequency** is given by:

$$\omega_0 = \sqrt{\frac{k}{m}} \tag{1.7}$$

The **damping ratio** is given by:

$$\zeta = \frac{c}{2\sqrt{mk}} = \frac{c}{2m\omega_0} \tag{1.8}$$

The damping ratio determines the system's behavior:

- **Underdamped** ($\zeta < 1$): Oscillatory response with exponential decay
- **Critically damped** ($\zeta = 1$): Fastest non-oscillatory return to equilibrium
- **Overdamped** ($\zeta > 1$): Slow non-oscillatory return to equilibrium

1.2.3 Analytical Solution

Each situation can be solved analytically:

- Overdamped: $\Rightarrow\ y(t) = Ae^{-\omega_0(\zeta+\sqrt{\zeta^2-1})t} + Be^{-\omega_0(\zeta-\sqrt{\zeta^2-1})t}$
- Critically damped: $\Rightarrow\ y(t) = (A + Bt)e^{-\omega_0 t}$
- Underdamped: $\Rightarrow\ y(t) = e^{-\zeta\omega_0 t}\left(A\cos\omega_d t + B\sin\omega_d t\right)$

where $\omega_d = \omega_0\sqrt{1-\zeta^2}$ is the **damped natural frequency**, and constants A, B are determined by initial conditions.

1.3 Burgers' Equation

1.3.1 Problem Description

Burgers' equation is a fundamental nonlinear partial differential equation that arises in fluid dynamics, gas dynamics, and heat conduction. It combines nonlinear advection with linear diffusion:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2} \tag{1.9}$$

where:

- $u(x,t)$ is the displacement variable $[m/s]$
- $\nu > 0$ is the viscosity coefficient $[m^2/s]$
- x is the spatial coordinate $[m]$
- t is the temporal coordinate $[s]$

The equation is supplemented with an **initial condition**: $u(x,0) = u_0(x)$.

1.3.2 Physical Interpretation

The three terms in Burgers' equation have clear physical meanings:

- $\frac{\partial u}{\partial t}$: **Time derivative** — rate of change of the solution
- $u\frac{\partial u}{\partial x}$: **Advection** — nonlinear transport of the solution field (sharpening effects)
- $\nu\frac{\partial^2 u}{\partial x^2}$: **Diffusion** — smoothing due to viscosity (dissipation)

The balance between advection (nonlinear steepening) and diffusion (smoothing) creates complex dynamics:

- Small ν: Advection dominates, steep fronts form and can develop shocks
- Moderate ν: Interplay between smoothing and transport
- Large ν: Diffusion dominates, smoothing then dissipation

1.3.3 Benchmark Initial Conditions

Three benchmark initial conditions are commonly used for testing Burgers' equation:

Gaussian Initial Condition

$$u(x,0) = \exp\left(-\frac{x^2}{2}\right) \tag{1.10}$$

This represents a smooth, localized disturbance that is transported and/or diffused over time.

Step Up Function

$$u(x,0) = \begin{cases} 1 & \text{if } x > 0 \\ 0 & \text{otherwise} \end{cases} \tag{1.11}$$

This represents a sharp discontinuity that smooths out and/or shows rarefraction over time.

N-Wave

$$u(x,0) = \begin{cases} -x & \text{if } |x| < 1 \\ 0 & \text{otherwise} \end{cases} \tag{1.12}$$

This represents a linear wave-like structure that produces two lobes that evolve towards each other, producing a shockwave for small viscosities.

Chapter 2

Understanding a Given Solution

As a first exercise, we were given a list of questions to answer thoroughly. These questions pertain a given PINN implementation for the damped harmonic oscillator. The questions provide a better understanding in the inner workings of the script and possible limitations and errors that are present. These are separated into questions about the methods and about the code.

Questions about the methods

Methods Question 1

Identify the total loss function in the code and write its expression down in your report. Identify the different contributions to the loss function.

$y_pred = model(t_obs_t)l_data = torch.mean((y_pred - y_obs_t) * *2)$
if $use_physics : l_phys = loss_physics(model, t_col_t)l_ic = loss_ic(model, t_ic_t)loss_total = l_data + LAMBDA_PHYS * l_phys + LAMBDA_IC * l_icelse : l_phys = torch.zeros(1, device = DEVICE)l_ic = torch.zeros(1, device = DEVICE)loss_total = l_data$

Methods Question 2

Is the inclusion of the initial condition loss essential for this particular example? When would it be essential?

Methods Question 3

The loss function used here is based on the mean squared errors (MSE) between predictions and data points or initial condition, and on the mean squared residual (MSR) of the PDE for the collocation points. Why are these specific functional forms used, and would there be alternative functions that could replace the MSE or MSR?

Methods Question 4

What is the type of neural network used in this example? How many input nodes, output nodes and hidden layers does it have? How many parameters are then being optimized? What type of activation function is used and why? What quantity or quantities are provided at the input, and what quantity or quantities do we extract from the output of the network? Are the dimensions of this Neural Network architecture adequate, too small or too large for this example? How would you determine this?

Methods Question 5

How many input samples are provided to the neural network model during each of its training cycles or epochs? What kind of samples are provided? Does the network receive again for each epoch the same exact samples? Would there be a better way to provide samples to this network for training over the various epochs?

Methods Question 6

The example uses the Adam algorithm as the so-called optimizer. What is the purpose of this optimizer? What is specific about the Adam algorithm, and what are its main hyper-parameters? Are there viable alternatives for the Adam optimizer?

Methods Question 7

The example also uses an additional optimizer: a learning rate scheduler. What does this scheduler do? Is it a necessary ingredient for training a neural network? What are its main hyper-parameters? Are there alternatives that can be usefully applied in this example?

Methods Question 8

How many epochs is the network trained for? How would you determine whether you have trained for too little or too many epochs?

Methods Question 9

Would it make sense to have a set of validation points? How would these look like? How would you use them?

Questions about the code

Code Question 1

Why are these two lines of code good, or bad, for:

```
torch.manual_seed(42)
np.random.seed(42)
```

Code Question 2

What is the purpose of the `np.ndarray` statements in this function definition:

```
def analytic(t: np.ndarray) -> np.ndarray:
```

Code Question 3

The data points are created randomly and then sorted before being offered to the neural net, using this statement:

```
t_obs = np.sort(np.random.uniform(0.1, T_TRAIN, N_OBS))
```

Is this good or bad?

Code Question 4

The collocation points are created uniformly on a fixed grid, using this statement:

```
t_col = np.linspace(0.0, T_EXTRAP, N_COL)
```

Is this good or bad?

Code Question 5

What is this statement doing? Is it necessary?

```
optimiser.zero_grad()
```

Code Question 6

What are these statements doing in the training cycle? Is their order important?

```
loss_total.backward()
optimiser.step()
scheduler.step()
```

Code Question 7

Why do the collocation time input tensor and the initial condition time input tensor require the gradient option, while the input data tensors do not require a gradient?

```
t_obs_t = to_tensor(t_obs)
y_obs_t = to_tensor(y_obs)
t_col_t = to_tensor(t_col, requires_grad=True)
t_ic_t = to_tensor(t_ic, requires_grad=True)
```

Code Question 8

When presenting arrays of time points to the input of my neural network, I need to convert them into a PyTorch tensor, using this helper function:

```
def to_tensor(arr, requires_grad=False):
    return torch.tensor(arr, dtype=torch.float32,
                        requires_grad=requires_grad).unsqueeze(1)
```

What does the `unsqueeze(1)` method of the `torch.tensor` class do, and why do I need to do this?

Code Question 9

When computing the first and second derivatives of $y(t)$ we use the `torch.autograd` method. Explain why the `grad_outputs` are set to ones and why we need to take the zero-th element [0] of this output in the statements below:

```
dy = torch.autograd.grad(
    y_hat, t,
    grad_outputs=torch.ones_like(y_hat),
    create_graph=True)[0]
```

Chapter 3

Structure and Implementation